PD-Disaggregation in Large Language Models

Chengxuan Pei, 225040508

Thesis Proposal

Final Report

School of Data Science

The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Large language model (LLM) serving is shaped by an unusual systems tension: the same model request contains a computation-intensive prefill phase and a memory-bandwidth-intensive decode phase. The prefill phase processes the prompt in parallel and builds the key-value (KV) cache; the decode phase generates tokens autoregressively while repeatedly reading and extending that cache. This split creates a design question at the heart of modern serving systems: should prefill and decode be aggregated on the same GPU workers, or should they be disaggregated onto separate execution resources?

In this survey, we study that question through three recent systems: Mooncake, DynaServe, and TaiChi. We place them in the broader lineage of LLM serving work, including continuous batching in Orca [1], memory management in vLLM/PagedAttention [2, 3], chunked prefill in Sarathi-Serve [4], and phase separation in DistServe and Splitwise [5, 6]. Mooncake presents a KVCache-centric disaggregated architecture for production LLM serving, emphasizing global prefix-cache reuse, asynchronous KV transfer, and cache-aware scheduling [7]. DynaServe argues that static aggregation and static disaggregation both fail under dynamic prompt-output mixtures, and introduces micro-requests plus two-level scheduling to adapt split points and batches at runtime [8]. TaiChi compares aggregation and disaggregation under different time-to-first-token (TTFT) and time-per-output-token (TPOT) service-level objectives (SLOs), then proposes a hybrid architecture based on latency shifting across prefill/decode phases and across requests [9].

Our central conclusion is that PD-disaggregation should not be treated as a binary choice. Mooncake shows that disaggregation becomes powerful when KV cache management is

promoted to a first-class system objective. DynaServe shows that request partitioning can be finer than the classical prefill/decode boundary. TaiChi shows that aggregation and disaggregation are endpoints in a tunable design space governed by TTFT, TPOT, and goodput. Together, these systems suggest that future LLM serving should be cache-centric, workload-adaptive, and explicitly SLO-aware.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation

LLM serving is different from ordinary neural-network inference because contemporary decoder-only transformer generation proceeds through two phases with sharply different execution patterns. The prefill phase consumes the input prompt, runs the prompt tokens through stacked self-attention and feed-forward layers, and materializes the KV cache. The decode phase then generates one token at a time, appending new KV states and repeatedly attending over the accumulated context. From a systems perspective, prefill resembles a high-parallelism matrix-matrix workload, while decode resembles a low-parallelism matrix-vector workload dominated by memory traffic [8, 9]. This two-phase structure is the reason why serving systems have evolved from general continuous batching [1] and memory-aware execution [3, 2] toward phase-aware and cache-aware designs [4, 5, 6, 7].

This asymmetry is not a small implementation detail. It determines how GPUs are batched, how memory is reserved, how requests queue, and how users experience latency. A serving platform must satisfy TTFT, which captures responsiveness before the first generated token, and TPOT or TBT, which captures smoothness during token streaming [7, 8, 9]. If TTFT is poor, the user waits before the model appears to respond. If TPOT or TBT is poor, the model starts quickly but streams slowly or unevenly. A system that maximizes

raw throughput while violating either latency objective cannot be considered effective for interactive serving.

The prefill-decode (PD) disaggregation problem asks how a serving system should place and schedule these two phases. PD aggregation colocates prefill and decode on the same GPU instance. This can improve utilization because all instances can execute mixed batches, but the two phases interfere with each other. PD disaggregation separates the phases across different workers or pools. This reduces interference and permits independent scaling, but it can fragment resources when the workload is unbalanced [8, 9]. In this survey, we study this tension from complementary viewpoints: Mooncake emphasizes KV cache as the central resource, DynaServe emphasizes elastic request partitioning, and TaiChi emphasizes SLO-dependent hybridization.

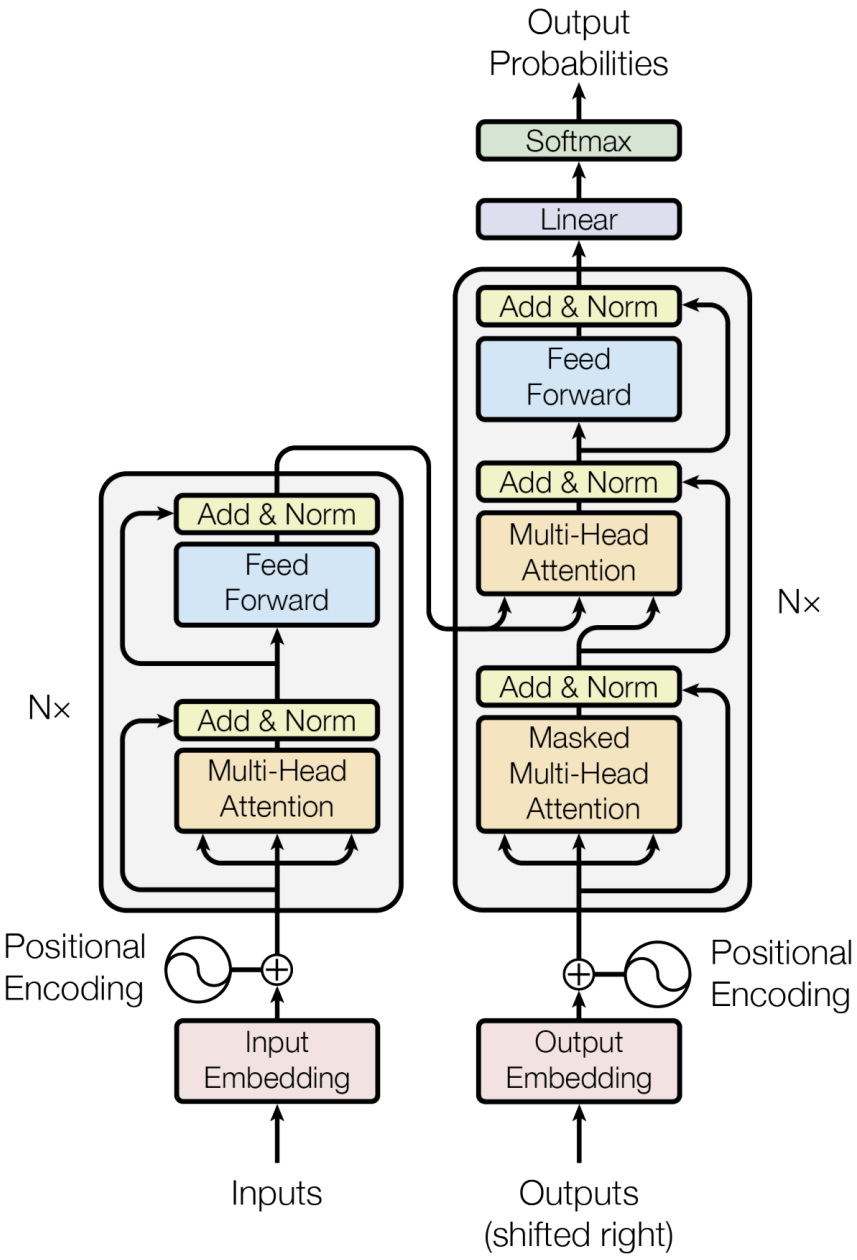


Figure 1.1: *Decoder-only transformer inference separates naturally into prefill and decode phases.*

1.2 Why PD Disaggregation Matters

The cost of serving LLMs is dominated by GPU infrastructure. The papers repeatedly use goodput, capacity, TTFT, TPOT, and TBT to connect system design to service cost and

user experience [7, 8, 9]. Goodput is especially important because it counts only work that satisfies latency SLOs. This distinction matters: a system can process many requests per second but still be unacceptable if the P99 TBT is too high or if a large fraction of requests miss TTFT.

PD aggregation and PD disaggregation expose different failure modes. In aggregation, a long prompt can introduce heavy prefill computation into a batch that also contains latency-sensitive decode steps. The decode steps then wait behind prefill work, causing high TPOT or TBT [8, 9]. Chunked prefill softens this problem by dividing the prompt into smaller pieces, but a static chunk size cannot match every workload shape [8, 9]. In disaggregation, decode can be isolated from prefill interference, but the system must transfer KV cache from prefill workers to decode workers. It must also choose the right prefill/decode resource ratio; otherwise, one side queues while the other side is idle [7, 8].

Recent work therefore shifts the question from "aggregation or disaggregation?" to "what granularity and policy should govern aggregation and disaggregation?" Mooncake answers at the cache-system level. It separates prefill and decode while building a distributed KV cache pool and a scheduler that exploits prefix reuse [7]. DynaServe answers at the request level. It allows each request to be split into micro-requests at token boundaries, so execution can resemble aggregation, disaggregation, or a hybrid depending on predicted cost and current load [8]. TaiChi answers at the SLO-policy level. It shows that aggregation and disaggregation win under different TTFT/TPOT regimes, then proposes latency shifting as a way to use slack from some requests to help requests at risk of SLO violation [9].

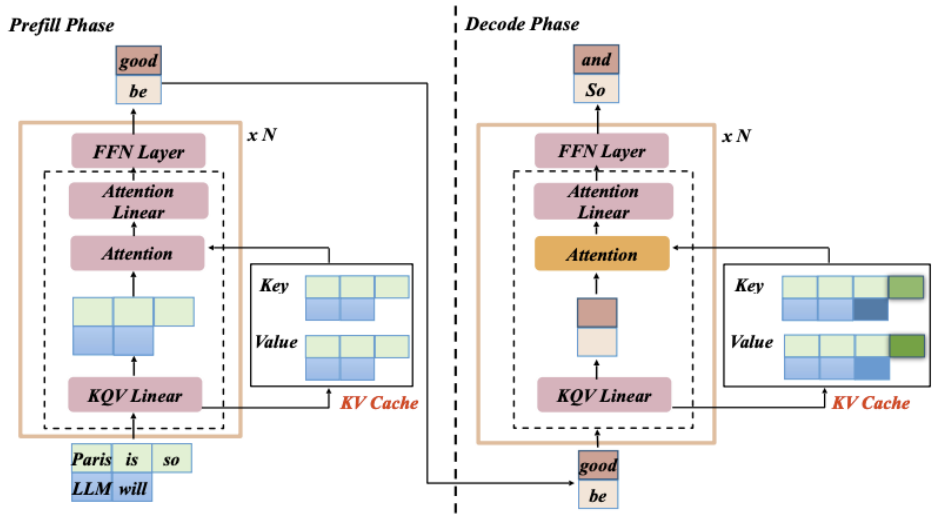


Figure 1.2: *Prefill is high-parallelism and compute-bound, while decode is low-parallelism and memory-bandwidth-bound.*

1.3 Report Scope

We organize the survey around PD-disaggregation and its neighboring serving techniques. The technical sequence moves from LLM inference basics to system-level comparison:

1. Introduce transformer inference, two-phase generation, and the KV cache.
2. Explain PD aggregation, PD disaggregation, and the trade-off between throughput and latency-SLO attainment.
3. Analyze Mooncake as a KVCache-centric disaggregated architecture.
4. Analyze DynaServe as a unified elastic executor based on micro-requests.
5. Analyze TaiChi as a hybrid aggregation-disaggregation system based on latency shifting.
6. Synthesize the common design lessons and open questions.

The remainder of the report is organized accordingly. Chapter 2 presents the background problem. Chapter 3 studies the three systems. Chapter 4 compares them along architecture,

scheduling, KV cache handling, SLO objective, and workload assumptions. Chapter 5 discusses broader implications. Chapter 6 concludes. The appendix records supplementary comparison material.

Chapter 2

Background: The PD Serving Problem

2.1 Two-Phase LLM Inference

Recent LLM serving papers describe online inference as a two-phase process. During prefill, the serving engine processes all prompt tokens and creates the initial KV cache. Because many prompt tokens can be processed together, the phase has substantial parallelism and is dominated by dense computation. During decode, generation proceeds token by token. Each token depends on the prior context, so the model repeatedly reads the KV cache and appends new entries. Decode is therefore constrained by memory bandwidth and by the length of the growing context [8, 9].

This distinction explains why a single GPU scheduling policy is hard to tune. Larger batches and larger prefill chunks improve arithmetic intensity for prefill, but they can delay decode tokens. Smaller chunks and decode-friendly batches reduce tail TPOT or TBT, but they may underuse compute during prefill. The three papers study different ways to resolve this mismatch without sacrificing either user experience or GPU efficiency.

2.2 The KV Cache as a System Resource

The KV cache is the bridge between prefill and decode. It records attention keys and values for previous tokens, allowing decode to reuse prior computations rather than recomputing

the entire prompt. Its benefit is essential for autoregressive generation, but its size and movement are costly. The longer the context, the more memory decode must read and the more state must move if prefill and decode are separated.

Earlier serving systems already identified memory management as a central problem. PagedAttention and vLLM manage KV cache blocks to reduce fragmentation and improve throughput on a single serving pool [3, 2]. Mooncake extends this line of thinking from memory allocation to cluster-level cache orchestration: its main conceptual move is to treat KV cache as the center of the architecture [7]. Rather than viewing the cache as a local by-product of one request, Mooncake builds a distributed KVCache pool using underexploited CPU, DRAM, SSD, and NIC resources around the GPU cluster. This pool enables prefix cache reuse across requests and supports transfer between prefill and decode instances. Mooncake's workflow is: load reusable prefix cache, run incremental prefill, stream generated KV cache to the decode side, then decode through continuous batching [7].

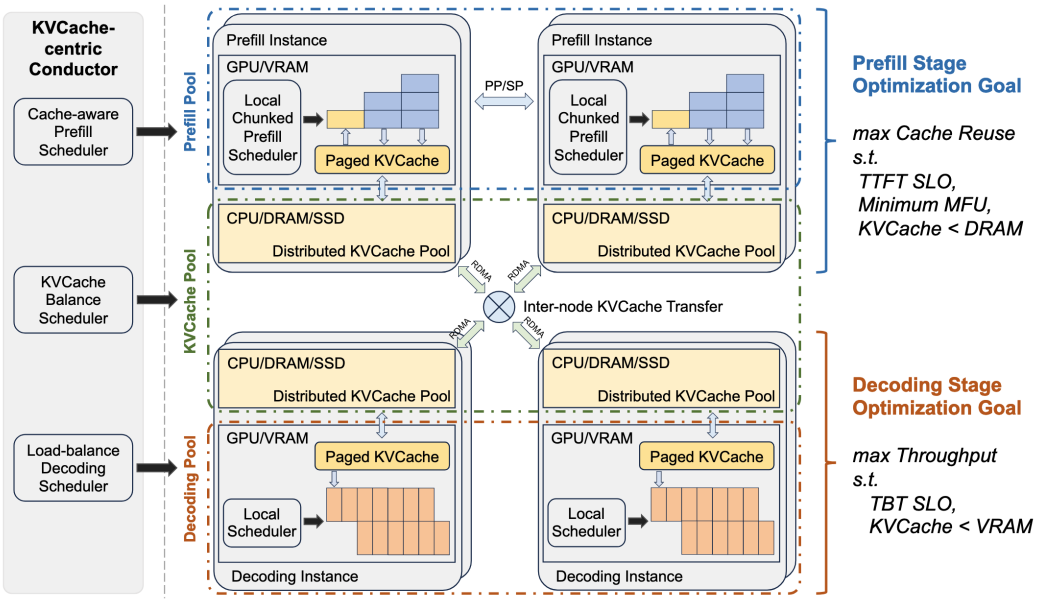


Figure 2.1: *Mooncake architecture, emphasizing separated prefill/decode pools and a distributed KV cache [7].*

2.3 Aggregation

PD aggregation colocates prefill and decode on the same GPU instance. The advantage is that all instances can participate in all work. This gives high resource utilization in some regimes and helps TTFT because the whole GPU pool can process incoming prompts. TaiChi explains that aggregation performs well when TTFT is tight and TPOT is relatively relaxed [9]. Under such a service objective, keeping all instances available for prefill can be better than dedicating only a subset to prefill.

The weakness is interference. Prefill and decode share the same instance, the same working memory, and the same batching opportunities. Continuous batching in Orca improves utilization by allowing requests to join and leave batches dynamically [1]; vLLM further makes such serving practical through efficient KV memory management [2, 3]. However, a heavy prefill chunk can still delay decode iterations already in progress. Sarathi-Serve mitigates this problem by chunking prefill and piggybacking decode work with chunked prefill computation [4]. DynaServe describes the remaining problem as prefill-decode interference and shows that chunked prefill can only partially control it [8]. TaiChi further analyzes the source of high TPOT in aggregation: decode latency increases as prefill computation is mixed into decode batches, and the interference becomes visible under balanced or TPOT-sensitive SLOs [9].

2.4 Disaggregation

PD disaggregation physically separates the prefill and decode phases. Prefill instances process prompts and produce the first token plus KV cache. Decode instances receive the KV cache and generate the remaining tokens. This architecture reduces direct interference because decode instances no longer need to share their execution stream with large prefill chunks. It also permits different resource ratios for prefill-heavy and decode-heavy workloads [7, 8, 9].

DistServe and Splitwise are representative phase-level disaggregation systems: they

separate prefill and decode onto different resources to eliminate interference and enable phase-specific optimization [5, 6]. The weakness is resource imbalance. Static disaggregation assumes that the chosen ratio of prefill to decode capacity will match the workload. DynaServe shows that this assumption often breaks under real traces because prompt and output lengths vary over time [8]. When prompts are long and outputs are short, prefill can bottleneck while decode resources idle. When outputs are long, decode can bottleneck while prefill resources idle. TaiChi describes a related limitation: disaggregation improves TPOT under strict TPOT constraints, but it can hurt TTFT because fewer instances handle prefill [9].

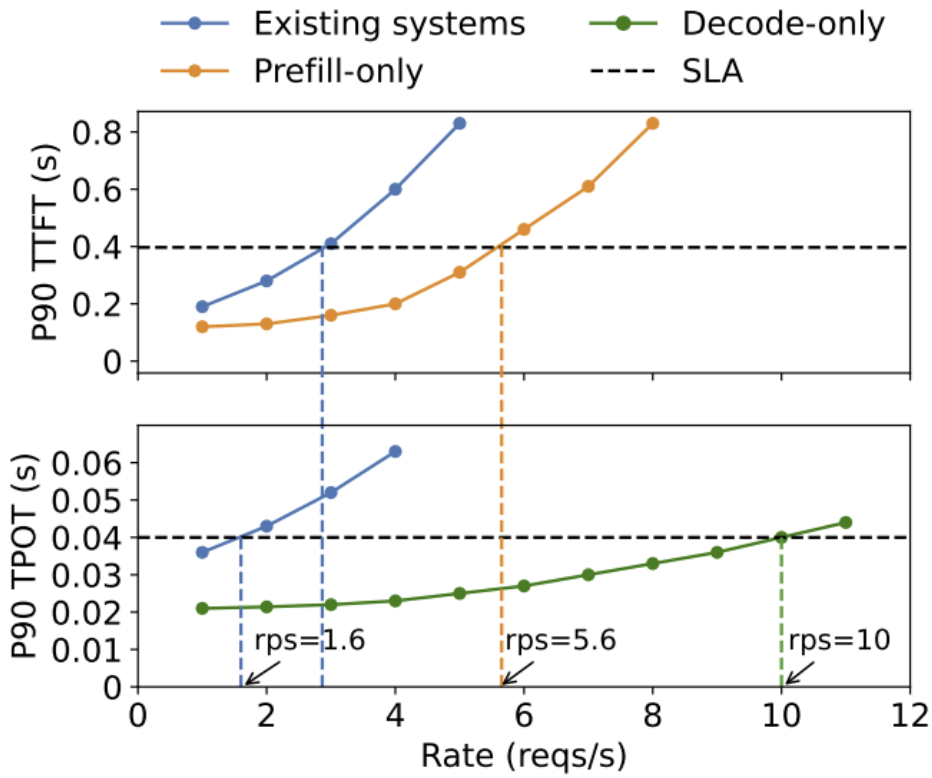


Figure 2.2: *PD aggregation and the latency metrics TTFT and TPOT/TBT.*

2.5 Workload Dynamics

The DynaServe paper places particular emphasis on workload imbalance. It describes real-world traces where prompt and output lengths vary persistently and temporally. Some intervals are prefill-heavy; others are decode-heavy; and the mix can change over time [8]. This makes both static colocation and static disaggregation fragile. Colocation can violate TBT during heavy prefill periods, while disaggregation can underutilize GPUs when the selected pool ratio does not match the current request mix.

Workload datasets such as BurstGPT make this issue concrete by exposing real request-arrival and prompt/output distributions [10]. Long-context serving systems such as LoongServe also show that context length can stretch the serving problem beyond the assumptions of a single-GPU or fixed-ratio scheduler [11]. The TaiChi paper frames workload variation through SLO regimes. If TTFT is tight and TPOT is relaxed, aggregation can be optimal. If TPOT is tight and TTFT is relaxed, disaggregation can be optimal. If both TTFT and TPOT are balanced, both policies can fail: aggregation suffers high TPOT from interference, and disaggregation suffers high TTFT from insufficient prefill processing capacity [9]. This observation motivates hybrid architectures.

2.6 Metrics

The surveyed papers use a shared vocabulary of serving metrics:

Table 2.1: *Serving metrics used by the surveyed PD-disaggregation literature.*

Metric	Meaning in the surveyed literature
TTFT	Time to first token. It measures responsiveness and is primarily affected by prompt processing, prefill queuing, and prefill execution [9].

TPOT/TBT	Time per output token or time between tokens. It measures streaming smoothness and is primarily affected by decode scheduling, KV-cache access, and prefill-decode interference [7, 8, 9].
Throughput	Raw completed work per unit time. It can be high even when latency SLOs are violated [8].
Goodput	Work completed while satisfying SLO constraints. The papers prefer goodput/capacity because it connects serving design to usable performance and cost [8, 9].
Capacity	The maximum request rate or useful token rate the system can sustain under SLO constraints; Mooncake and DynaServe both report large capacity improvements over baselines [7, 8].

2.7 Design Space

The three works reveal a layered design space. At the architecture layer, a system can colocate phases, separate phases, or combine specialized instance types. At the cache layer, a system can keep KV cache local, transfer it between phases, or build a distributed reusable cache pool. At the scheduling layer, a system can use static batching, chunked prefill, request-level phase assignment, token-boundary micro-request partitioning, or latency shifting. At the objective layer, a system can optimize raw throughput, TTFT, TPOT/TBT, or goodput under both constraints.

Mooncake, DynaServe, and TaiChi occupy different parts of this design space. Mooncake is the clearest cache-centric disaggregation system. DynaServe is the clearest fine-grained elastic partitioning system. TaiChi is the clearest SLO-regime-aware hybrid system. Around them, Orca, vLLM, Sarathi-Serve, DistServe, Splitwise, FastServe, NanoFlow, BurstGPT, and LoongServe provide the broader survey context for batching, memory management, chunking, phase splitting, scheduling, workload characterization, and long-context serv-

ing [1, 2, 3, 4, 5, 6, 12, 13, 10, 11]. The following chapter studies the three focal systems in detail.

Chapter 3

Three Representative Systems

3.1 Mooncake: KVCache-Centric Disaggregation

Mooncake is the serving platform described for Kimi, an LLM chatbot service developed by Moonshot AI [7]. Its starting point is that long-context serving creates opportunities and pressure around the KV cache. Whereas vLLM/PagedAttention focuses on efficient memory management inside the serving engine [2, 3], Mooncake asks how a cluster can reuse and transfer KV cache across prefill and decode resources. Reused prefixes can save substantial prefill computation, but only if the system can find, store, move, and schedule KV cache blocks efficiently. Mooncake therefore combines PD disaggregation with a distributed KVCache pool, cache-aware global scheduling, and an optimized transfer engine.

Mooncake's workflow can be summarized in four stages. First, if a prefix cache exists, the system loads reusable KV cache blocks from remote CPU memory into GPU memory. Second, the prefill node executes incremental prefill and stores newly generated KV cache back into CPU memory. Third, the KV cache transfer is executed asynchronously and overlapped with incremental prefill, streaming cache blocks from each model layer to the destination decode node. Fourth, once the cache is received at the decode side, the request enters continuous batching for token generation.

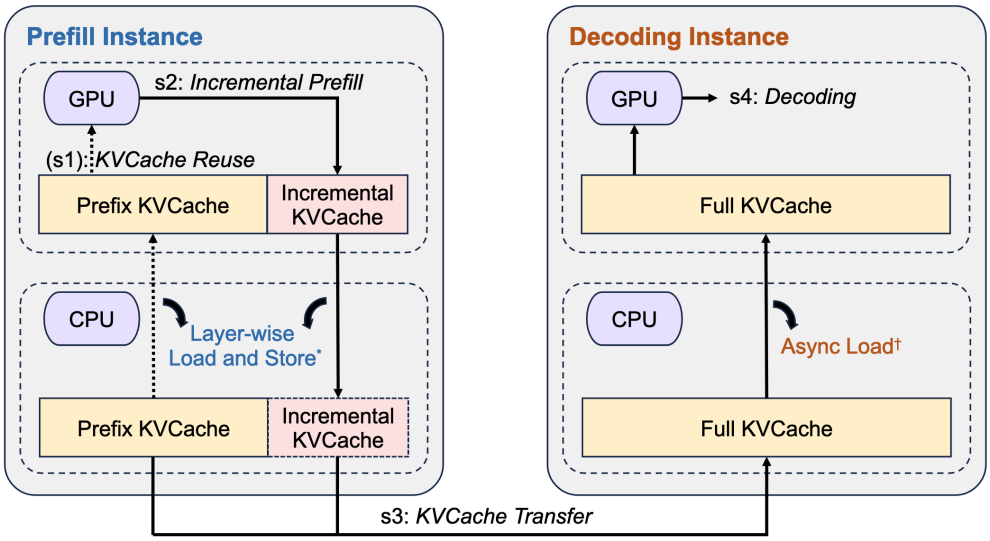


Figure 3.1: *Mooncake workflow, showing prefix reuse, incremental prefill, cache transfer, and decode [7].*

3.1.1 Architecture

Mooncake separates the cluster into prefill and decode resources, but the separation is not merely a static P/D split. The architecture includes a KVCache-centric conductor, local schedulers, chunked prefill schedulers, a distributed KVCache pool, and an RDMA-based transfer engine [7]. The prefill-side objective is to maximize cache reuse subject to TTFT and resource constraints. The decode-side objective is to maximize throughput subject to TBT and VRAM constraints. This explicit split of objectives is important: Mooncake recognizes that prefill and decode optimize different latency and resource dimensions.

The distributed KVCache pool uses resources around the GPU cluster, including CPU memory, storage, and network bandwidth [7]. This is a practical design choice because GPU memory is scarce and expensive, while CPU and storage resources may be comparatively underutilized. By placing reusable cache blocks outside GPU memory and transferring them when needed, Mooncake trades storage and network movement for avoided prefill computation.

3.1.2 Scheduling

Mooncake's scheduler is cache-aware. For each request, it must choose a prefill instance and a decode instance while considering prefix cache reuse, TTFT, TBT, load balance, and cache placement [7]. The scheduling problem is more complex than ordinary load balancing because a request may be cheaper on a particular prefill node if that node can access relevant cache blocks or if the cache transfer path is favorable. The Mooncake paper also discusses overload handling, hotspot migration, and prediction-based early rejection as mechanisms to protect SLOs under pressure [7].

The important lesson is that cache placement changes the meaning of load. A lightly loaded GPU may not be the best target if moving cache to it is expensive or if another node has reusable prefix state. Conversely, a node with useful cache may be valuable even if its compute load is not minimal. Mooncake therefore expands serving scheduling from "where is compute available?" to "where can the system best combine cache reuse, transfer, and phase execution?"

3.1.3 Evaluation Claims

The Mooncake paper reports that Mooncake is effective for long-context workloads. In real-trace experiments, it increases effective request capacity by 59% to 498% compared with baseline methods while complying with latency SLOs [7]. In deployment, Mooncake is described as operating across thousands of nodes and processing more than 100 billion tokens daily; the paper reports that the architecture lets Kimi handle 115% more requests on NVIDIA A800 clusters and 107% more on H800 clusters compared with previous systems [7]. The evaluation covers ArXiv Summarization, L-Eval, simulated data, and real workload latency distributions.

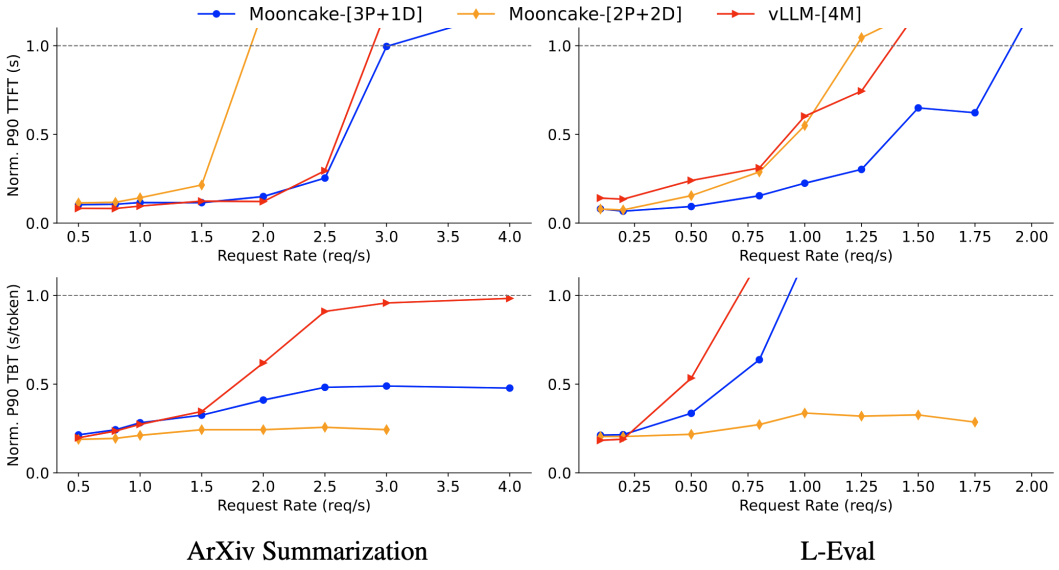


Figure 3.2: *Mooncake end-to-end evaluation results [7].*

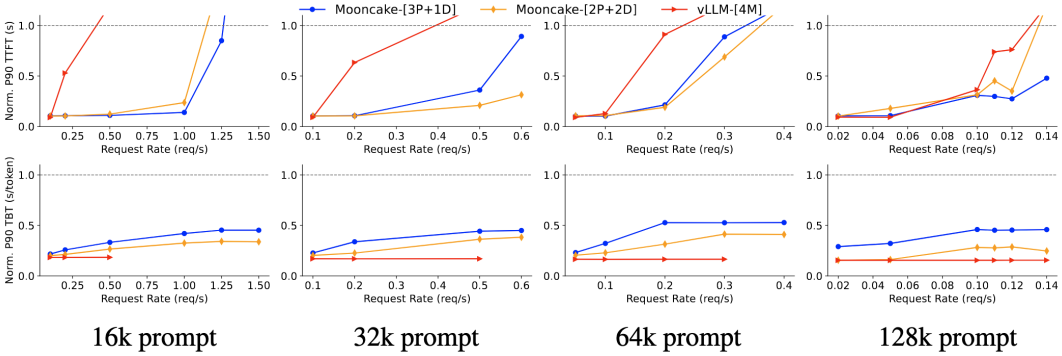


Figure 3.3: *Additional Mooncake evaluation results [7].*

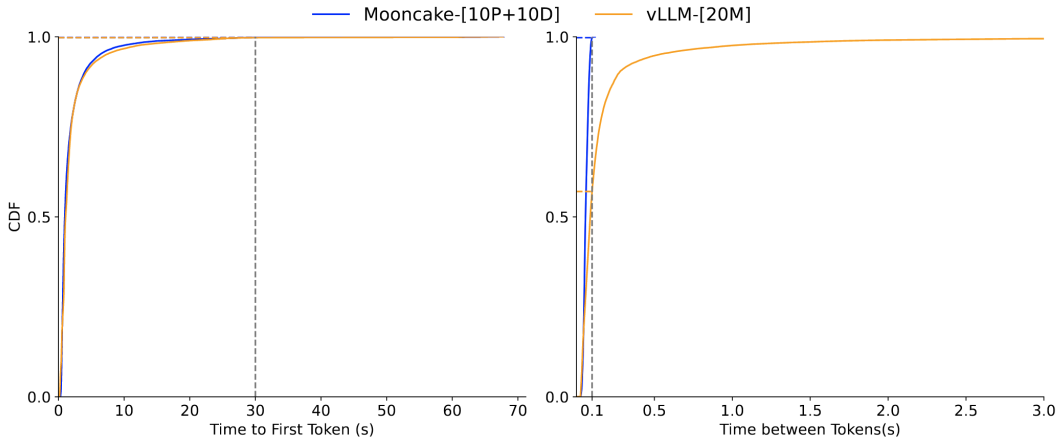


Figure 3.4: *Mooncake latency distributions for TTFT and TBT under long-context workloads [7].*

3.2 DynaServe: Unified and Elastic Execution

DynaServe begins from a different critique: both colocated serving and statically disaggregated serving fail under dynamic, unbalanced workloads [8]. Colocation inherits the lineage of Orca-style continuous batching and vLLM-style efficient KV allocation [1, 2, 3]; chunked prefill systems such as Sarathi-Serve reduce decode stalls by breaking prefill into smaller pieces [4]. However, colocation can still violate TBT because heavy prefill work delays decode. Disaggregation systems such as DistServe and Splitwise can satisfy TBT by isolating the phases [5, 6], but they may underutilize GPUs because the fixed prefill/decode split does not match the workload. DynaServe addresses this gap through dynamic PD batching.

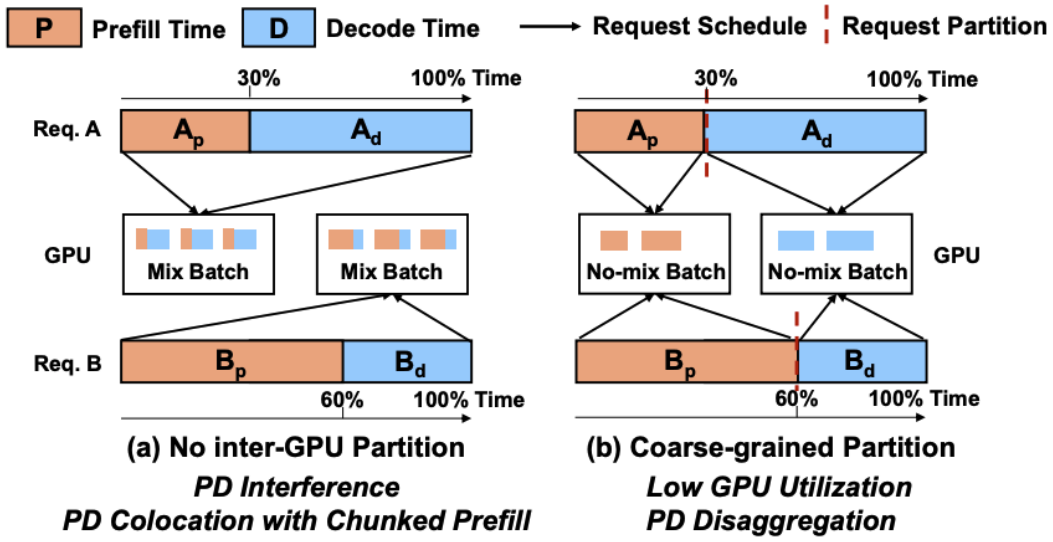


Figure 3.5: *DynaServe motivation, contrasting colocation, disaggregation, and dynamic PD batching [8].*

3.2.1 Micro-Request Abstraction

DynaServe's key abstraction is the micro-request. A request with prompt length and output length can be split at an arbitrary token boundary into at most two contiguous segments [8]. These segments may correspond to prefill, decode, or a mixture of both. This is more flexible than classical PD disaggregation, which splits only at the boundary between prefill and decode, and more flexible than chunked prefill, which primarily splits long prompts.

This abstraction lets DynaServe emulate several policies. If no split is useful, the request can remain on one instance, similar to aggregation. If the prefill/decode boundary is the best split, the system can behave like disaggregation. If a hybrid split is better, part of the request can be routed differently to balance current load. The micro-request abstraction therefore turns PD placement into a continuous scheduling decision rather than a fixed architecture.

3.2.2 Two-Level Scheduling

DynaServe uses a global scheduler and local schedulers [8]. The global scheduler receives requests, predicts execution behavior, selects split points, and routes micro-requests to

unified GPU instances. The local scheduler on each GPU instance builds token-level batches under SLO-aware controls. It considers batch size, prefill-to-decode token ratio, decode context length, HBM limits, and observed latency behavior.

The two-level structure separates global placement from local batching. The global scheduler needs a fast approximation because every request can have many possible split points, and the system cannot search the full combinatorial space. The local scheduler then adapts to actual runtime conditions, widening batches when latency allows and reducing interference when TBT approaches the SLO. DynaServe also uses chunked KV cache transfers so that micro-requests spanning different instances can cooperate through cache movement [8].

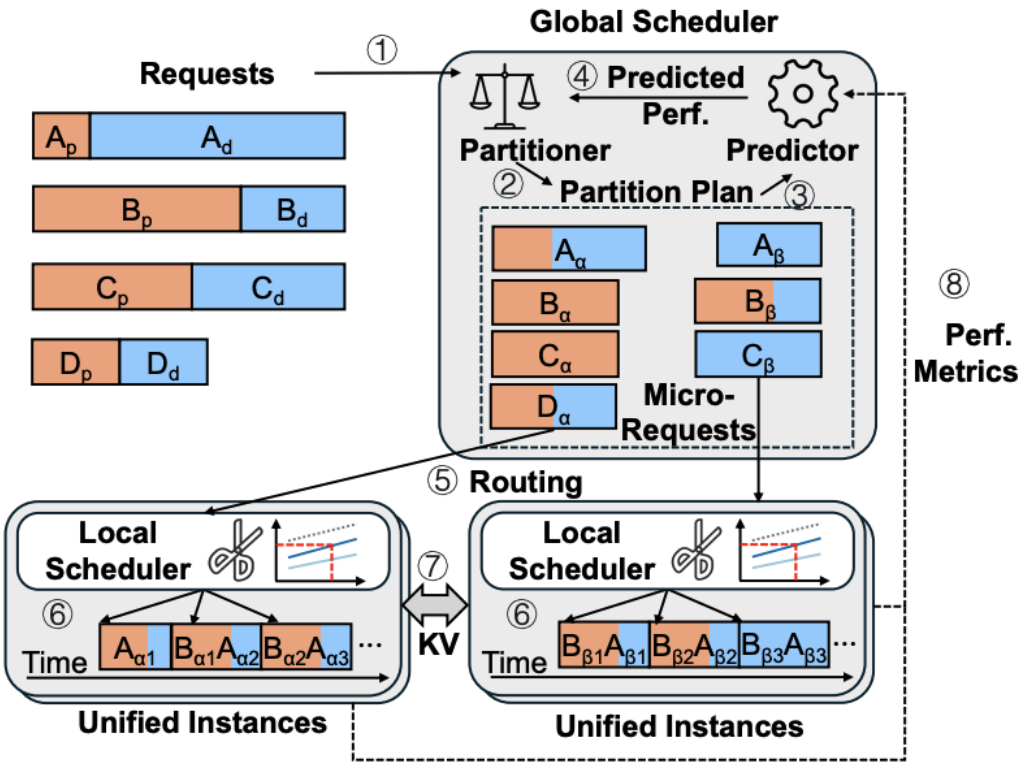


Figure 3.6: *DynaServe workflow: global scheduler, local schedulers, micro-request routing, and KV transfer [8].*

3.2.3 Evaluation Claims

The DynaServe paper reports improvements across real-world traces and workload mixes, including BurstGPT-style dynamic workloads and Azure-code-style request patterns [8, 10]. It states that DynaServe boosts serving capacity from 1.15× to 3.07×, improves goodput by up to 1.91× over colocated baselines and up to 1.61× over disaggregated baselines, and improves performance by up to 60% in a hybrid workload under SLO [8]. The central evaluation claim is not only that DynaServe is faster, but that it maintains high SLO attainment while improving utilization under changing prompt/output distributions.

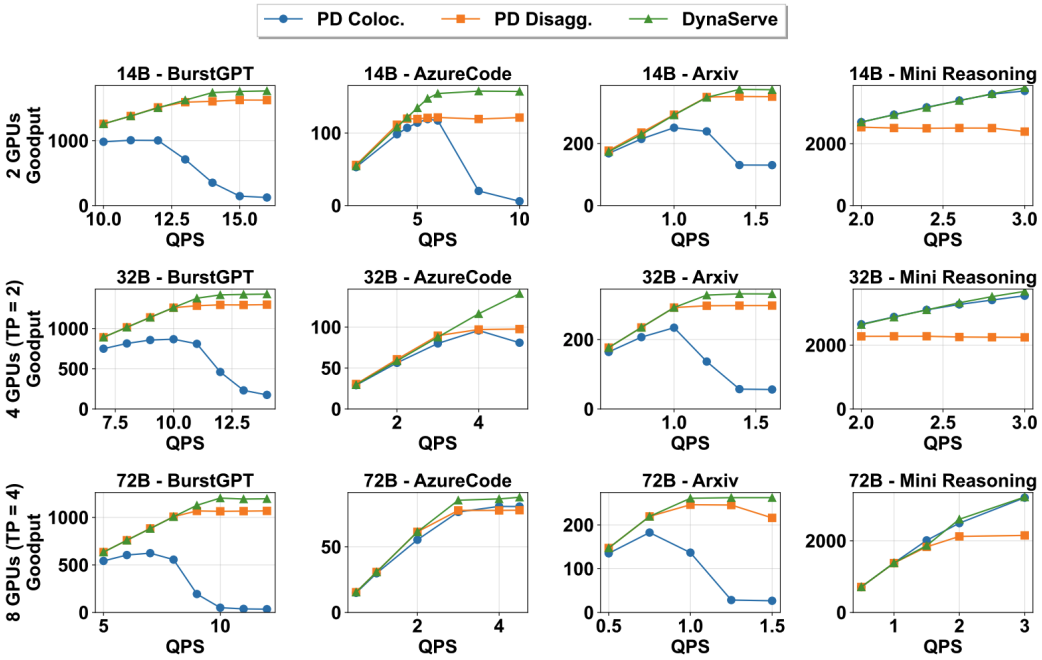


Figure 3.7: *DynaServe goodput and serving-capacity comparisons [8].*

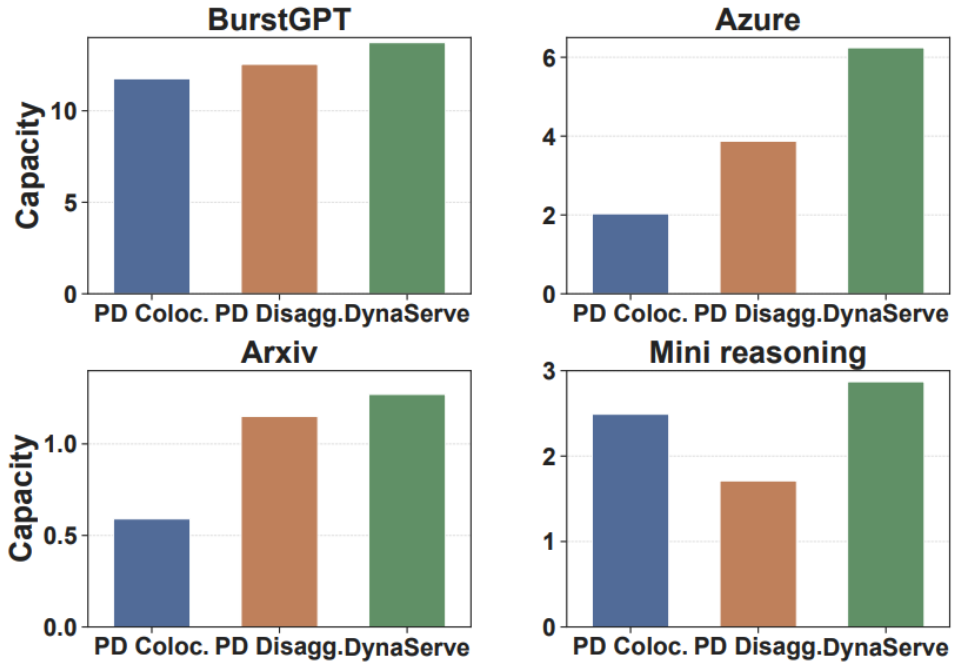


Figure 3.8: *DynaServe serving-capacity comparison across BurstGPT, Azure, ArXiv, and Mini Reasoning workloads [8].*

3.3 TaiChi: Hybrid Aggregation-Disaggregation

TaiChi directly asks whether prefill-decode aggregation or disaggregation is superior [9]. Its answer is conditional. In the aggregation lineage, systems such as Orca, FastServe, NanoFlow, and Sarathi-Serve improve batching, scheduling, overlap, or chunking while keeping the two phases colocated [1, 12, 13, 4]. In the disaggregation lineage, DistServe and Splitwise separate the phases to remove interference [5, 6]. TaiChi argues that neither lineage dominates globally. PD aggregation is best when TTFT is tight and TPOT is relaxed, because all GPU instances can help with prompt processing. PD disaggregation is best when TPOT is tight and TTFT is relaxed, because decode benefits from isolation. Under balanced TTFT and TPOT constraints, neither pure policy is optimal: aggregation violates TPOT due to interference, while disaggregation violates TTFT due to limited prefill processing capacity [9].

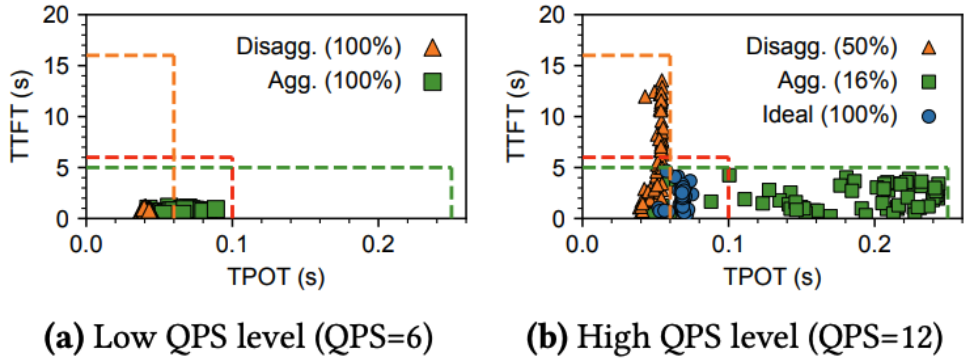


Figure 3.9: *TaiChi observation: aggregation and disaggregation perform differently under TTFT/TPOT SLO regimes [9].*

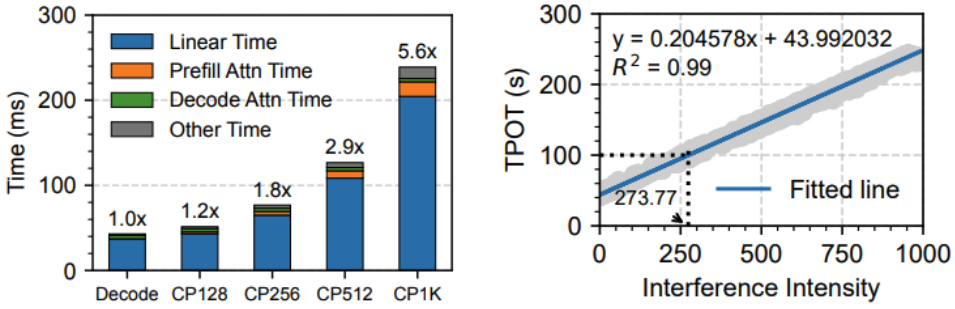


Figure 3.10: *TaiChi microbenchmark showing how prefill interference increases decode latency [9].*

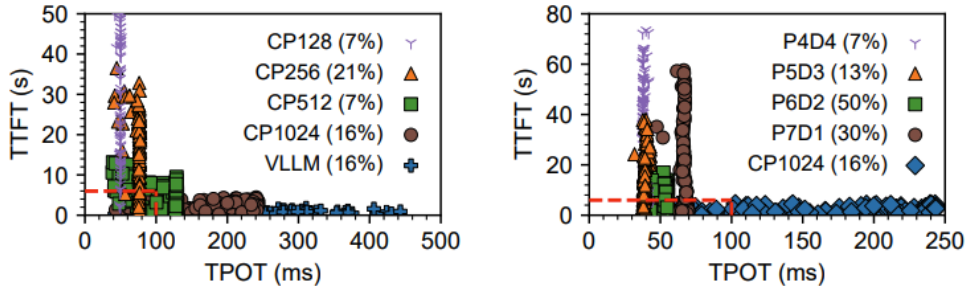


Figure 3.11: *TaiChi comparison of chunked-prefill and P/D-heavy configurations in the TTFT/TPOT space [9].*

3.3.1 Latency Shifting

TaiChi's key idea is latency shifting. Rather than treating every request's latency budget independently, TaiChi uses slack from requests that are safely within SLO to help requests

at risk of violating SLO [9]. This is a scheduling idea: if one request can tolerate a slower decode or prefill without missing its SLO, the system can move resources toward another request that urgently needs them. TaiChi applies this across both phases and across requests.

This idea requires architectural support. TaiChi uses prefill-heavy (P-heavy) instances and decode-heavy (D-heavy) instances [9]. P-heavy instances are tuned for fast prefill but can impose higher interference on decode. D-heavy instances are tuned for low-interference decode but process prefill more slowly. TaiChi exposes controls over the ratio of instance types and chunk sizes, allowing the system to move along the spectrum from aggregation-like behavior to disaggregation-like behavior.

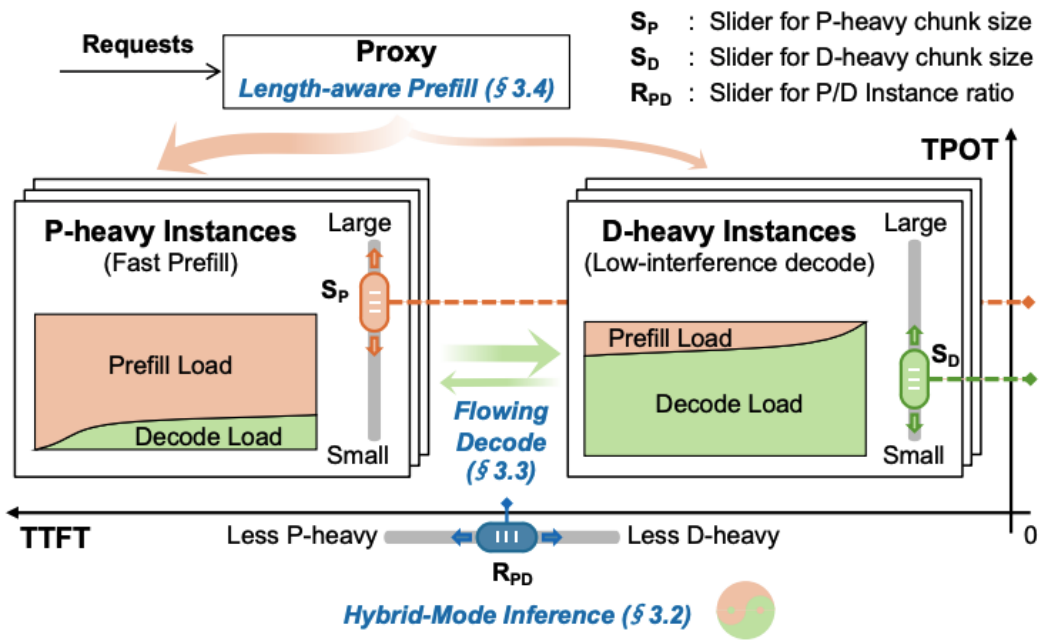


Figure 3.12: *TaiChi hybrid-mode architecture, with P-heavy and D-heavy instances, adjustable chunk sizes, and a tunable P/D instance ratio [9].*

3.3.2 Flowing Decode Scheduling

TaiChi's flowing decode scheduling controls TPOT by dynamically migrating decode requests between D-heavy and P-heavy instances [9]. The mechanism has three steps. First, decode begins on D-heavy instances to keep TPOT low. Second, TaiChi uses a longest-first degradation policy to offload selected decode requests to P-heavy instances; these are

requests with longer observed output so far and more opportunity to absorb controlled degradation. Third, if a migrated request approaches the TPOT SLO, TaiChi flows it back to a D-heavy instance.

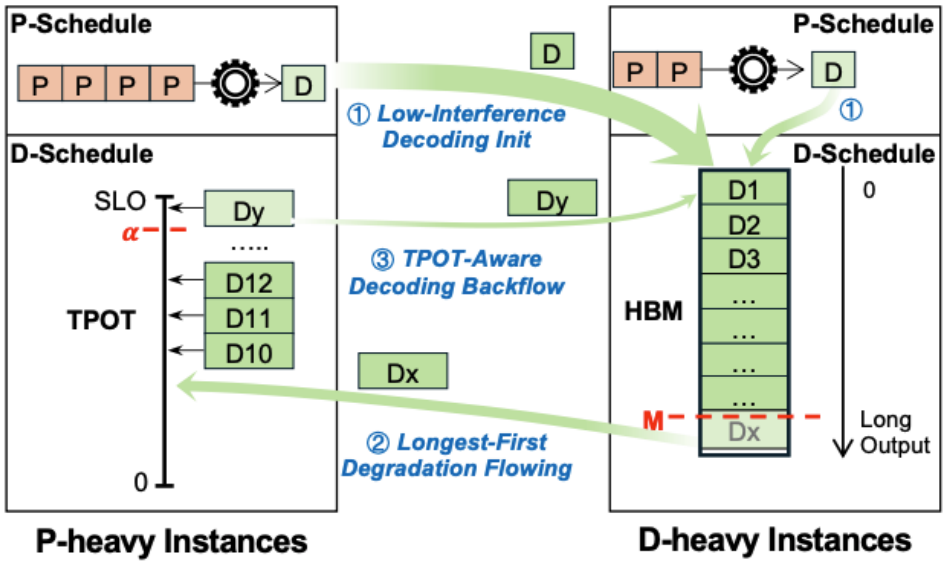


Figure 3.13: *Flowing decode scheduling: decode requests can move between D-heavy and P-heavy instances according to TPOT slack [9].*

3.3.3 Length-Aware Prefill Scheduling

TaiChi's length-aware prefill scheduling controls TTFT by exploiting the fact that short prefill requests may have enough slack to run on slower D-heavy instances [9]. The scheduler estimates waiting and execution time on both instance types and routes a request to the instance with fewer queuing prefill tokens when doing so will not violate TTFT. This shifts P-heavy capacity toward long or urgent prefill requests.

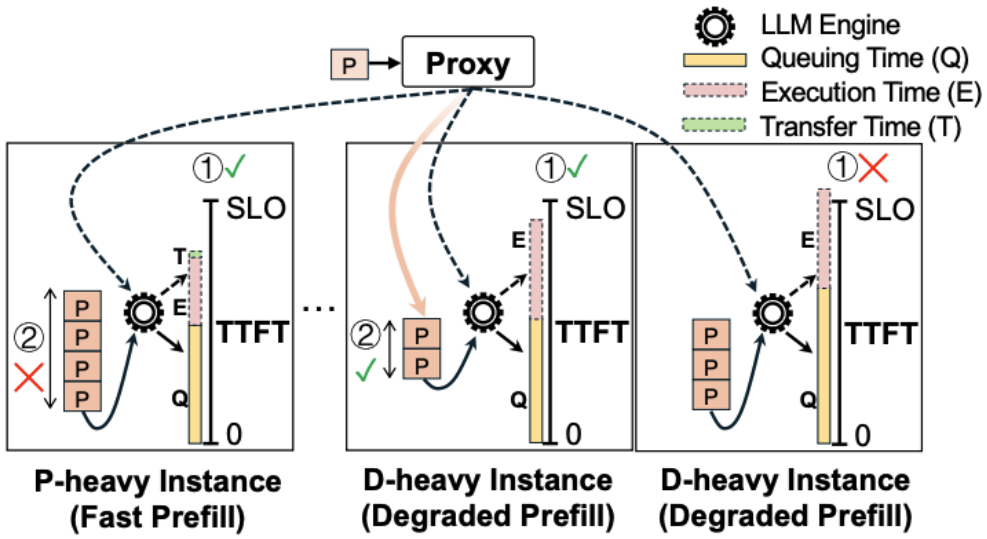


Figure 3.14: *Length-aware prefill scheduling: short prefill work may be routed to D-heavy instances when TTFT slack permits [9].*

3.3.4 Evaluation Claims

TaiChi reports goodput improvements of up to 77% over state-of-the-art systems under balanced TTFT and TPOT SLOs [9]. It also reports large TTFT and TPOT reductions relative to pure disaggregation and pure aggregation in the settings studied. The important point for this report is the shape of the result: TaiChi is not claiming that hybrid execution always looks the same. It adapts to SLO regimes. When TTFT dominates, it can move toward aggregation. When TPOT dominates, it can move toward disaggregation. When both matter, it uses hybrid mode and latency shifting.

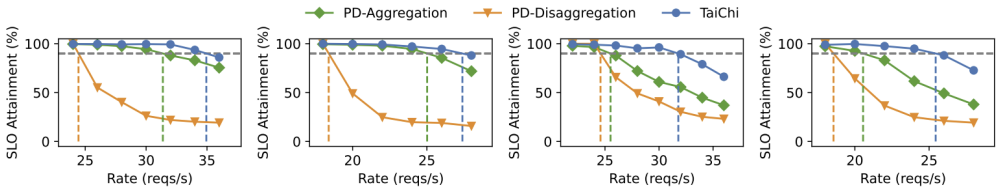
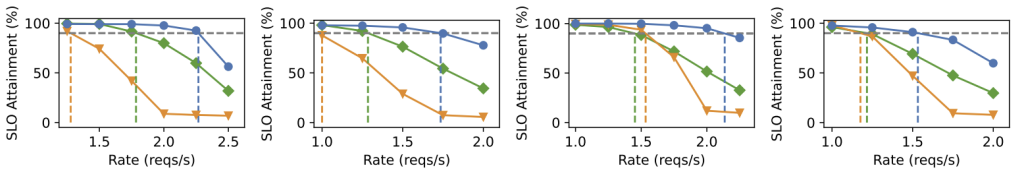


Figure 3.15: *TaiChi SLO-attainment results under SLO1 and SLO2 for 14B and 32B models [9].*



(a) 14B under SLO1 **(b)** 32B under SLO1 **(c)** 14B under SLO2 **(d)** 32B under SLO2

Figure 3.16: *Additional TaiChi SLO-attainment results at lower request rates [9].*

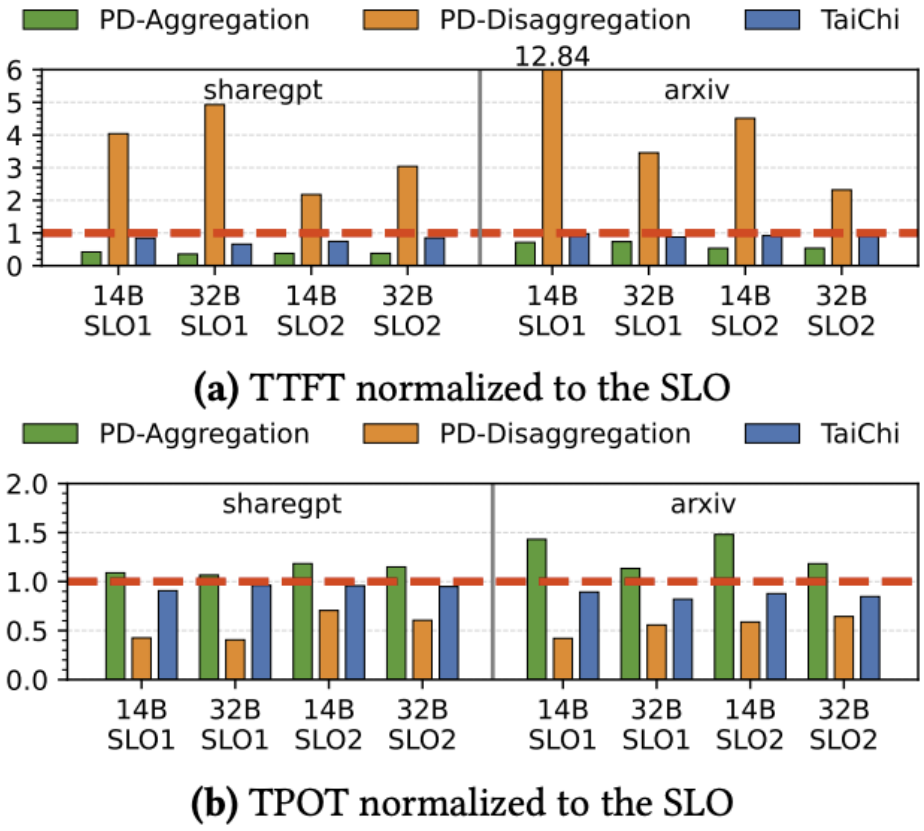


Figure 3.17: *TaiChi normalized TTFT and TPOT relative to the configured SLOs [9].*

Chapter 4

Comparative Analysis

4.1 From Binary Choice to Continuum

The surveyed systems form a progression. Early serving designs established the importance of continuous batching and efficient KV memory management [1, 3, 2]. Chunked prefill then reduced aggregation-side interference [4], while phase-splitting systems showed the value of separating prefill and decode [5, 6]. Mooncake assumes that PD disaggregation is valuable for long-context serving when the system can make KV cache reuse and transfer efficient [7]. DynaServe challenges the rigidity of fixed disaggregation by allowing token-boundary request partitioning across unified instances [8]. TaiChi challenges the aggregation-versus-disaggregation debate itself by showing that the best policy depends on TTFT and TPOT SLOs [9]. Together, they transform the design question from a binary architecture decision into a continuum of phase placement, cache placement, request partitioning, and SLO-aware scheduling.

4.2 Architecture Comparison

Table 4.1: *Architectural comparison of Mooncake, DynaServe, and TaiChi.*

System	Architectural center	Main mechanism	Primary lesson
Mooncake	Disaggregated prefill/decode clusters plus distributed KVCache pool	Prefix cache reuse, incremental prefill, asynchronous KV transfer, cache-aware scheduling	KV cache should be managed as a global serving resource [7].
DynaServe	Unified GPU instances executing micro-requests	Adaptive request partitioning, global split selection, local SLO-aware batching, chunked KV transfer	The split boundary can be chosen dynamically rather than fixed at the phase boundary [8].
TaiChi	Hybrid aggregation-disaggregation with P-heavy and D-heavy instances	Latency shifting, flowing decode scheduling, length-aware prefill scheduling	Aggregation and disaggregation are SLO-dependent endpoints of a broader hybrid space [9].

Mooncake's architecture is most explicit about non-GPU resources. It uses CPU, DRAM, SSD, and NIC resources around the GPU cluster to build a distributed KV cache [7]. DynaServe's architecture is most explicit about GPU fungibility. It treats instances as unified workers that can execute different micro-request segments [8]. TaiChi's architecture is most explicit about differentiated GPU roles. It intentionally creates P-heavy and D-heavy instances, then routes requests according to latency budget [9].

4.3 Scheduling Comparison

The scheduling policies also differ in granularity. Mooncake schedules whole requests across prefill and decode instances, but the decision is strongly informed by cache reuse and

transfer. DynaServe schedules micro-requests, choosing split points within a request and batching token-level work locally. TaiChi schedules latency slack, moving decode requests and prefill work between specialized instance types.

This reveals three meanings of adaptivity:

1. **Cache adaptivity**: Mooncake adapts to where reusable prefixes and hot cache blocks are located.
2. **Workload adaptivity**: DynaServe adapts to time-varying prompt/output mixtures and current GPU load.
3. **SLO adaptivity**: TaiChi adapts to the relative tightness of TTFT and TPOT and to per-request slack.

4.4 KV Cache Comparison

KV cache is present in all three systems, but it has different roles. In Mooncake, the KV cache is the organizing principle. The architecture is explicitly KVCache-centric, and the paper emphasizes global cache hit rate, transfer engine performance, prefix reuse, and cache-aware scheduling [7]. In DynaServe, KV cache is the state that allows micro-requests on different instances to cooperate; chunked KV transfer supports fine-grained partitioning [8]. In TaiChi, KV cache transfer is part of the broader ability to separate prefill and decode across P-heavy and D-heavy instances, but the paper's main conceptual focus is latency shifting rather than cache storage [9].

4.5 SLO Comparison

The systems emphasize different SLO surfaces. Mooncake focuses on latency-related SLOs such as TTFT and TBT while maximizing effective throughput and request capacity [7]. DynaServe focuses strongly on P99 TBT and goodput under dynamic workloads [8]. TaiChi

explicitly models the two-dimensional space of TTFT and TPOT, arguing that the optimal policy changes depending on which constraint is tight [9].

4.6 Failure Modes

Each focal system identifies a different failure mode in prior systems. Mooncake targets the cost of repeated prefill for long-context and shared-prefix workloads. Without global cache reuse, repeated prompts waste computation; without optimized transfer, disaggregation can be slowed by KV movement [7]. DynaServe targets load imbalance under dynamic workloads, including workloads similar to those captured by BurstGPT [8, 10]. A static disaggregated ratio can leave GPUs idle while another phase queues, and chunked colocation can still cause latency spikes despite Sarathi-style mitigation [4, 8]. TaiChi targets balanced SLO failure. Pure aggregation and pure disaggregation each excel in one SLO regime but fail when TTFT and TPOT are both important [9].

4.7 Common Design Principle

Despite different mechanisms, the common design principle is to stop treating GPU compute as the only scarce resource. Mooncake treats cache and transfer bandwidth as scarce resources. DynaServe treats token-boundary partitioning and local batch composition as scarce scheduling opportunities. TaiChi treats latency slack as a resource that can be reallocated. In all three cases, better goodput comes from managing a richer set of resources than raw GPU utilization.

Chapter 5

Discussion and Outlook

5.1 Lesson 1: Cache Is King

The first major lesson is that cache is king. Mooncake provides the strongest evidence for this lesson. Long-context prompts and repeated prefixes make prefill computation expensive, but also reusable. If the serving system can locate prefix cache blocks, move them efficiently, and schedule requests around them, it can trade storage and network resources for reduced computation [7]. This is a natural fit for production clusters where GPU memory is precious but surrounding CPU memory, storage, and network resources can be organized into a larger cache substrate.

Cache-centric design also changes how PD disaggregation should be judged. A naive disaggregated design may look unattractive if it simply adds KV transfer overhead. Mooncake shows that when KV movement is optimized and overlapped, disaggregation can become a vehicle for global reuse and higher capacity [7]. The relevant question is not just whether prefill and decode are separated, but whether the cache lifecycle is designed around that separation.

5.2 Lesson 2: Flexibility Matters

The second major lesson is that static partitioning cannot handle dynamic workloads. DynaServe makes this point most directly. Workloads contain long prompts, short prompts, long outputs, short outputs, and temporal bursts. A serving system fixed to one architecture must either overprovision one side or accept SLO failures. Micro-requests provide one way to recover flexibility by allowing each request to be split according to predicted cost and current load [8].

DynaServe's approach is important because it expands the scheduling granularity. Traditional PD disaggregation assumes a request has one natural boundary: prefill before decode [5, 6]. DynaServe treats the token sequence itself as the scheduling space [8]. That perspective is useful even beyond the specific implementation: as context lengths grow and request shapes diversify, future systems may need increasingly fine-grained ways to reshape work without making the scheduler too slow. This complements the long-context concerns raised by LoongServe and Mooncake, where sequence length amplifies both scheduling and cache-management pressure [11, 7].

5.3 Lesson 3: Balance Is Achievable

The third major lesson is that hybrid architectures can extract the best of both worlds. TaiChi gives the most explicit argument. It observes that pure aggregation and pure disaggregation each win under a different SLO regime, then introduces a hybrid mode for balanced SLOs [9]. The important novelty is not merely mixing two instance types. It is the use of latency shifting: requests with slack can be intentionally degraded, within their SLO budget, so that endangered requests receive more favorable resources.

This lesson reframes latency. Latency is usually considered a constraint, but TaiChi treats unused latency budget as a schedulable resource. A request does not need the absolute fastest possible execution if it is already safely within TTFT and TPOT. That slack can be used to improve system-wide goodput. This logic is especially attractive for services

with diverse applications because different interactions may have different tolerance for first-token delay and token-stream speed.

5.4 Open Problems

The three works point toward several open problems.

5.4.1 Prediction Under Uncertainty

DynaServe uses prediction to choose request split points, and TaiChi notes that output length prediction is imperfect [8, 9]. This matters because decode length is unknown at arrival time. If the system predicts too short an output, it may allocate too little decode capacity; if it predicts too long an output, it may overprotect a request that did not need special handling. Future systems need prediction methods that are fast enough for online scheduling and robust enough under distribution shifts.

5.4.2 Transfer and Cache Placement

Mooncake demonstrates that optimized transfer makes cache-centric disaggregation practical [7]. DynaServe depends on chunked KV transfer between unified instances [8]. TaiChi relies on the ability to place prefill and decode across specialized instance types [9]. These designs all depend on high-speed interconnects and efficient cache-block movement. Future work must decide when transfer is worth the benefit, how much cache to replicate, and how to avoid network hotspots.

5.4.3 Multi-Objective Scheduling

The papers jointly show that LLM serving is multi-objective. A scheduler must consider TTFT, TPOT/TBT, goodput, GPU utilization, cache hit rate, queueing delay, memory capacity, and transfer overhead. Optimizing any single metric can harm another. The challenge is to

design policies that are explainable, stable, and fast while still capturing the true economics of production serving.

5.4.4 From Cluster-Level to Application-Level SLOs

The surveyed systems focus on system-level metrics, but their motivation is application-level user experience. TTFT and TPOT are proxies for responsiveness and smooth streaming, while goodput captures the amount of useful work that survives those SLO constraints [5, 8, 9]. Future systems may need to adapt policies based on task type. A code-generation task, a summarization task, and a chatbot task may tolerate different TTFT/TPOT trade-offs. TaiChi's SLO-regime framing is a step in this direction [9].

5.5 Implications for Future Serving Systems

The synthesis of the three papers suggests several design principles for future PD-disaggregated LLM serving systems.

First, serving systems should expose the KV cache as a managed object. Cache blocks should have location, reuse likelihood, transfer cost, and scheduling value. Mooncake's global cache makes this idea concrete [7].

Second, serving systems should avoid hard-coded phase ratios. Workloads vary too much for a fixed prefill/decode split to remain optimal. DynaServe's micro-request abstraction and TaiChi's P-heavy/D-heavy controls both provide mechanisms for elasticity [8, 9].

Third, serving systems should optimize goodput rather than throughput alone. This is the shared metric that turns latency SLOs into usable capacity. A high-throughput system that violates TBT or TTFT does not serve the real objective.

Fourth, serving systems should treat slack as valuable. TaiChi's latency shifting shows that not every request needs minimum latency; many requests need only latency below the SLO [9]. Allocating the difference intelligently can raise overall goodput.

Chapter 6

Conclusion

In this survey, we synthesized PD-disaggregation in large language model serving through the lens of recent serving systems. We began from the transformer inference split between prefill and decode. Prefill is parallel and computation-heavy; decode is sequential and memory-bandwidth-heavy. This asymmetry makes LLM serving a problem of phase placement, KV cache management, batching, and SLO-aware scheduling.

Mooncake shows that PD disaggregation can be effective at production scale when KV cache is treated as a first-class distributed resource. Its KVCache-centric architecture uses global cache reuse, asynchronous transfer, and cache-aware scheduling to improve long-context serving capacity under latency SLOs [7]. DynaServe shows that fixed aggregation and fixed disaggregation are both too rigid for dynamic workloads. Its micro-request abstraction and two-level scheduling framework allow requests to be partitioned and batched according to predicted cost and current load [8]. TaiChi shows that the aggregation/disaggregation decision depends on TTFT and TPOT SLOs. Its hybrid architecture and latency-shifting mechanisms use P-heavy and D-heavy instances to improve goodput under balanced constraints [9].

Our combined conclusion is that PD-disaggregation is not a single architecture. It is a design space. The most promising systems are cache-centric, flexible, and SLO-aware. They manage KV cache as carefully as compute, adapt partitioning to workload dynamics, and

treat latency slack as a resource. In this sense, the progression from Mooncake to DynaServe to TaiChi reflects a broader trend in LLM serving: the field is moving from static phase separation toward adaptive orchestration of computation, memory, cache movement, and user-facing latency.

Appendix A

Supplementary Material

A.1 Figure Summary

The following figures summarize the main visual evidence and architectural diagrams used throughout the report.

Table A.1: *Summary of figures used in the survey.*

Figure	Role in this report
Fig. 1.1	Introduces the decoder-only transformer context and motivates the two-phase inference explanation.
Fig. 1.2	Shows the contrast between prefill and decode.
Fig. 2.1	Provides the Mooncake architectural picture [7].
Fig. 3.1	Shows the Mooncake request workflow from prefix-cache reuse through decode [7].
Fig. 3.2	Presents Mooncake evaluation material [7].
Fig. 3.4	Presents Mooncake TTFT and TBT latency distributions [7].
Fig. 3.5	Presents DynaServe's motivation around the latency-throughput frontier [8].
Fig. 3.6	Shows DynaServe's global and local scheduler workflow [8].

Fig. 3.8	Presents DynaServe serving-capacity comparisons across workload families [8].
Fig. 3.9	Summarizes TaiChi's observations on aggregation and disaggregation under different SLO regimes [9].
Fig. 3.10	Shows TaiChi's decode-interference microbenchmark [9].
Fig. 3.11	Compares TaiChi's configuration choices in the TTFT/TPOT space [9].
Fig. 3.12	Shows TaiChi's P-heavy and D-heavy architecture [9].
Fig. 3.13	Shows TaiChi's flowing decode scheduling mechanism [9].
Fig. 3.14	Shows TaiChi's length-aware prefill scheduling mechanism [9].
Fig. 3.17	Presents TaiChi normalized TTFT and TPOT results [9].

A.2 Claim Summary

Table A.2: *Summary of claims used in the survey.*

No.	Source	Claim used in the report
1	DynaServe; TaiChi	LLM inference has distinct prefill and decode phases with different resource characteristics [8, 9].
2	DynaServe; TaiChi	Prefill has high parallelism and is computation-bound, while decode is lower-parallelism and memory-bandwidth sensitive [8, 9].
3	Mooncake; DynaServe; TaiChi	TTFT and TPOT/TBT are key serving metrics for online LLM systems [7, 8, 9].
4	DynaServe; TaiChi	PD aggregation colocates prefill and decode but can create prefill-decode interference [8, 9].

5	Mooncake; TaiChi	DynaServe;	The serving architecture question is how to satisfy both prefill-side and decode-side latency objectives while improving goodput [7, 8, 9].
9	Mooncake		Mooncake is a KVCache-centric disaggregated architecture for LLM serving [7].
10	Mooncake		Mooncake separates prefill and decode clusters and uses a distributed KVCache [7].
11	Mooncake		Mooncake uses underexploited CPU, DRAM, SSD, and NIC resources around the GPU cluster for KV cache storage and transfer [7].
12	Mooncake		The Conductor/scheduler selects prefill and decode instances with cache reuse and SLOs in mind [7].
13	Mooncake		Mooncake's workflow includes prefix-cache loading, incremental prefill, KV transfer, and continuous-batching decode [7].
14	Mooncake		Mooncake reports 59% to 498% effective request-capacity improvement in real-trace tests under SLOs [7].
15	Mooncake		Mooncake is described as deployed across thousands of nodes and serving more than 100 billion tokens daily [7].
16	DynaServe		LLM inference systems must meet strict latency SLOs while maximizing goodput [8].
17	DynaServe		Real-world prompt and response length variability skews prefill and decode load [8].
18	DynaServe		Colocation with chunked prefill can violate latency SLOs under heavy prefill interference [8].

19	DynaServe	Static disaggregation can underutilize GPUs under mismatched stage loads [8].
20	DynaServe	DynaServe introduces micro-requests, splitting each request at token boundaries into at most two segments [8].
21	DynaServe	DynaServe uses a global scheduler for split selection and local schedulers for SLO-aware batching [8].
22	DynaServe	DynaServe uses chunked KV cache transfer for cross-instance micro-request execution [8].
23	DynaServe	DynaServe reports 1.15× to 3.07× serving-capacity improvement and up to 1.91× goodput improvement over colocated baselines [8].
24	DynaServe	DynaServe reports up to 1.61× goodput improvement over disaggregated baselines [8].
25	TaiChi	The best choice between aggregation and disaggregation depends on TTFT and TPOT SLOs [9].
26	TaiChi	Aggregation performs best under tight TTFT and relaxed TPOT [9].
27	TaiChi	Disaggregation performs best under relaxed TTFT and tight TPOT [9].
28	TaiChi	Under balanced TTFT and TPOT, neither pure aggregation nor pure disaggregation is optimal [9].
29	TaiChi	TaiChi uses P-heavy and D-heavy instances to unify aggregation and disaggregation [9].
30	TaiChi	TaiChi introduces latency shifting across phases and requests [9].

31	TaiChi	Flowing decode scheduling migrates decode requests between D-heavy and P-heavy instances [9].
32	TaiChi	Length-aware prefill scheduling routes short prefill requests to D-heavy instances when TTFT slack permits [9].
33	TaiChi	TaiChi reports up to 77% goodput improvement under balanced SLOs [9].

A.3 Additional Figures

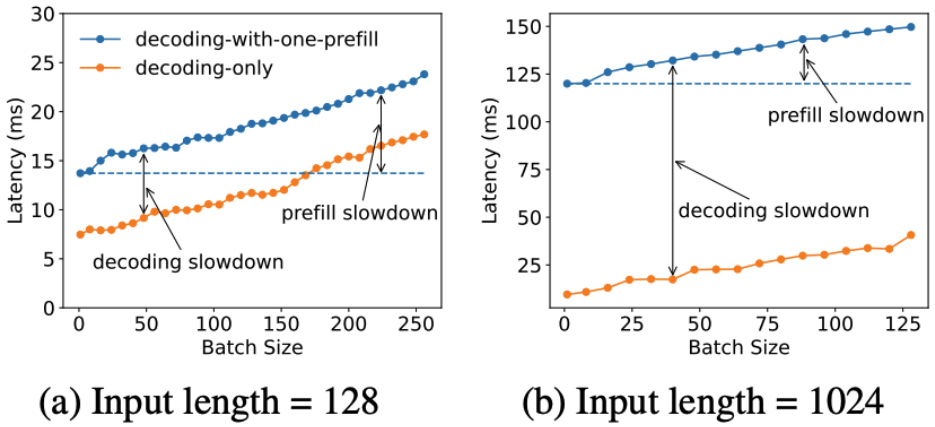


Figure A.1: *Prefill-decode interference in colocated execution.*

Metric		P-8192, D-32		P-2048, D-512		P-219, D-1467	
		Disagg.	Coloc.	Disagg.	Coloc.	Disagg.	Coloc.
MFU (%)	G1	43.24	38.94	30.59	21.26	2.09	13.98
	G2	0.19	38.94	7.93	21.26	14.34	13.98
HBM Usage (%)	G1	4.83	39.67	1.17	94.23	0	95.2
	G2	5.77	40.03	94.83	95.7	96.27	95.33
p50-TBT (ms)		**22.70**	309.68	**47.00**	**46.86**	**50.63**	**47.99**
p99-TBT (ms)		**58.09**	352.93	**65.11**	336.81	**74.47**	162.67
Throughput (rps)		0.83	**1.49**	2.56	**2.83**	1.68	**2.86**
Attainment (%)		**100.00**	1.73	**99.83**	84.09	**99.95**	94.46

Figure A.2: *DynaServe quantitative comparison of disaggregation and colocation under different prefill/decode load splits [8].*

A.4 Extended Comparison Notes

A.4.1 Mooncake Notes

Mooncake's contribution can be summarized as an operational claim: PD disaggregation becomes more valuable when cache reuse is integrated with scheduling. The paper does not merely separate the two phases. It rethinks the KV cache as a distributed object whose reuse saves prefill work and whose transfer must be hidden or minimized [7]. This is why Mooncake's workflow includes prefix-cache loading before incremental prefill and asynchronous cache streaming during prefill. The goal is to reduce waiting time at the decode side while keeping the prefill side efficient.

Mooncake's evaluation claims are especially tied to long-context serving [7]. Long contexts increase prefill cost and KV cache size. They therefore make prefix reuse more valuable and also make transfer efficiency more important. A design that ignores cache reuse may repeat computation; a design that ignores transfer may block decode; a design that integrates both can raise SLO-constrained capacity.

A.4.2 DynaServe Notes

DynaServe's contribution can be summarized as a granularity claim: fixed phase boundaries are too coarse for dynamic workloads [8]. Micro-requests allow the scheduler to split the logical token sequence at an arbitrary point. This means DynaServe is not committed to a single prefill/decode boundary. It can select no split, a phase split, or a hybrid split.

The global-local scheduler split is important because the scheduling search space is large [8]. A global scheduler must decide quickly where to cut and route requests. A local scheduler must then compose batches that satisfy the TBT SLO while using GPU resources effectively. The paper's design therefore combines prediction, runtime metrics, and local feedback.

A.4.3 TaiChi Notes

TaiChi's contribution can be summarized as an SLO-regime claim: aggregation and disaggregation are each optimal only under particular latency priorities [9]. The system's hybrid mode exists because balanced TTFT and TPOT require something neither endpoint provides. Latency shifting is the key abstraction. Instead of minimizing every request's latency, TaiChi seeks to maximize the number of SLO-satisfied requests by moving slack.

TaiChi's two scheduling mechanisms implement this idea at the decode and prefill sides [9]. Flowing decode scheduling lets decode requests move away from or back to low-interference resources depending on TPOT pressure. Length-aware prefill scheduling uses short-request slack to preserve P-heavy capacity for more urgent prefill work. Both mechanisms treat SLO slack as an allocatable resource.

References

[1] G.-I. Yu, J. S. Jeong, G.-W. Kim, S. Kim, and B.-G. Chun, "Orca: A distributed serving system for transformer-based generative models," in *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI 22)*, 2022, pp. 521–538.

[2] vLLM Project, "vllm: Easy, fast, and cheap llm serving for everyone," 2023.

[3] W. Kwon, Z. Li, S. Zhuang, Y. Sheng, L. Zheng, C. H. Yu, J. E. Gonzalez, H. Zhang, and I. Stoica, "Efficient memory management for large language model serving with pagedattention," in *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP)*, 2023, pp. 611–626.

[4] A. Agrawal, N. Kedia, A. Panwar, J. Mohan, N. Kwatra, B. S. Gulavani, A. Tumanov, and R. Ramjee, "Taming throughput-latency tradeoff in llm inference with sarathi-serve," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 117–134.

[5] Y. Zhong, S. Liu, J. Chen, J. Hu, Y. Zhu, X. Liu, X. Jin, and H. Zhang, "Distserve: Disaggregating prefill and decoding for goodput-optimized large language model serving," in *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, 2024, pp. 193–210.

[6] P. Patel, E. Choukse, C. Zhang, A. Shah, I. Goiri, S. Maleki, and R. Bianchini, "Splitwise: Efficient generative llm inference using phase splitting," in *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*, 2024, pp. 118–132.

[7] R. Qin, Z. Li, W. He, J. Cui, H. Tang, F. Ren, T. Ma, S. Cai, Y. Zhang, M. Zhang, Y. Wu, W. Zheng, and X. Xu, "Mooncake: A kvcache-centric disaggregated architecture for llm serving," *ACM Transactions on Storage*, 2025.

[8] C. Ruan, Y. Chen, D. Tian, Y. Shi, Y. Wu, J. Li, and C. Li, "Dynaserve: Unified and elastic execution for dynamic disaggregated llm serving," 2025.

[9] C. Wang, P. Zuo, Z. Chen, Y. Liang, Z. Yu, and M.-C. Yang, "Prefill-decode aggregation or disaggregation? unifying both for goodput-optimized llm serving," 2025.

[10] Y. Wang, Y. Chen, Z. Li, X. Kang, Z. Tang, X. He, R. Guo, X. Wang, Q. Wang, A. C. Zhou, and X. Chu, "Burstgpt: A real-world workload dataset to optimize llm serving systems," 2024.

[11] B. Wu, S. Liu, Y. Zhong, P. Sun, X. Liu, and X. Jin, "Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism," 2024.

[12] B. Wu, Y. Zhong, Z. Zhang, G. Huang, X. Liu, and X. Jin, "Fast distributed inference serving for large language models," 2023.

[13] K. Zhu, W. Zhao, H. Zhao, P. Zuo, Z. Wang, T. Zhang, D. Chen, Q. Yang, Y. Zhou, and Y. Ruan, "Nanoflow: Towards optimal large language model serving throughput," 2024.